

What is a String?

In C programming, a **string** is simply a sequential collection or sequence of characters.

Under the hood, C does not have a native "string" data type like modern languages do. Instead, a string is implemented as a **character array** that is terminated by a special character known as the **Null Character (\0)**.

The Rule of the Null Terminator: A character array is only recognized as a valid string if it concludes with the null character (\0).

The **ASCII value** of the null character \0 is exactly **0**.

In memory, the integer zero (0) is stored directly in that final array block rather than a text symbol.

String Initialization Styles

You can declare and initialize strings in multiple ways, each with slightly different compiler behaviors:

Style 1: Explicit Character List with Manual Null Terminator

```
char str1[] = {'B', 'H', 'O', 'P', 'A', 'L', '\0'};
```

Style 2: Direct String Literal (Most common and recommended)

```
char str2[] = "BHOPAL";
```

Style 3: Fixed-Size Allocation

```
char str3[10] = "BHOPAL"; ----> Allocates 10 bytes; remaining slots are filled with 0
```

Critical Initialization Traps

1. The Explicit List Trap

If you initialize a string character-by-character using single quotes, the compiler **does not** automatically add the null character. You must append '\0' manually, or printing functions will continue reading past your data, printing corrupted garbage values.

✗ `char str[] = {'M', 'i', 't', 't', 'h', 'u'}; ----> Missing null terminator!`

`char str[] = {'M', 'i', 't', 't', 'h', 'u', '\0'}; ----> Safe and valid.`

2. The Fixed-Size Border Crash

When specifying a fixed size for a string literal, you must account for the invisible '\0' character.

✗ `char str[5] = "hello";` ----> **Incorrect!** "hello" requires 5 slots for letters plus 1 slot for '\0'. Defining the size as 5 cuts off the null terminator, creating bugs.

`char str[6] = "hello";` ----> **Correct!**

Reading and Modifying String Elements

Because strings are arrays, you can access and modify single characters using their index positions (subscripts).

A. Accessing Elements

To examine a single character or print its underlying numerical ASCII integer value, match it with the correct format specifier:

Use **%c** to display the character symbol.

Use **%d** to display its raw ASCII integer code.

```
char str[] = "Physics Wallah";  
printf("%c\n", str[5]);  ----> Outputs: 'c'  
printf("%d\n", str[9]);  ----> Outputs: 97 (The ASCII value of 'a')
```

B. Modifying Elements

You can change characters inline by overwriting their indices using text literals or ASCII values:

```
char str[] = "Physics Wallah";
```

```
str[0] = 'M'; ----> Overwriting using a char literal  
String becomes: "Mhysics Wallah"
```

```
str[1] = 97; ----> Overwriting using an ASCII value ('a')  
String becomes: "Paysics Wallah"
```

Printing Strings

You can loop through a string character-by-character, or output the entire block instantly.

Method 1: Iterative Character Traversal

Instead of counting elements manually, you can set a loop to run until it encounters the null terminator `\0`:

```
#include <stdio.h>
int main()
{
    char arr[] = "Bhopal";
    int i = 0;

    while (arr[i] != '\0') ----> Loop terminates automatically when hitting the null byte
    {
        printf("%c", arr[i]);
        i++;
    }
    return 0;
}
```

Method 2: The %s Format Specifier (Bulk Output)

The `%s` specifier allows you to target and output an entire string at once. When using `%s`, you only need to pass the **array name** as a parameter.

```
char str[] = "BHOPAL";
printf("%s", str);
Outputs: BHOPAL
```

Shorthand Length Calculation

You can find a string's length by running a bodyless for loop that increments until `str[i]` hits the null terminator (which evaluates to 0/False):

```
char str[] = "BHOPAL";
int i;
for (i = 0; str[i]; i++); ----> Semicolon here means no inner loop body exists
printf("Length is %d", i);
Outputs: 6
```

Handling String Inputs

Reading strings from users requires careful selection of input functions.

1. Using scanf() (Single-Word Only)

```
char s1[10];  
scanf("%s", s1);
```

No & Needed: You do not use the address-of operator (&) for strings because the array name s1 automatically points to the base memory address of the block.

The Multi-Word Flaw: scanf() treats spaces, tabs, and newlines as hard termination boundaries. If you enter "Gaurav Kandpal", scanf() will stop at the space and store only "Gaurav".

💡 **Advanced Tip:** You can limit the number of characters scanf() reads by inserting a size modifier, such as scanf("%7s", name);, which safely captures only the first 7 characters.

2. Using gets() (Unsafe / Obsolete)

The gets() function was designed to read a whole line of input, including spaces.

```
char name[30];  
gets(name); ----> Reads an entire line
```

🚨 **Critical Vulnerability:** gets() completely ignores the boundaries of your array buffer. If a user types 50 characters into a 30-byte array, gets() will overwrite adjacent memory, leading to **Buffer Overflow** corruption or system crashes.

3. Using fgets() (The Modern, Secure Standard)

To address the safety gaps of scanf() and gets(), modern C development relies on fgets(). It enforces strict buffer protection by requiring you to explicitly declare a size limit.

Syntax Definition:

```
fgets (array_name , array_size , input_source);
```

```
char str[30];
```

```
printf("Enter your name: ");
fgets(str, 30, stdin); ----> stdin means take input from the standard keyboard
```

Mechanism: fgets() reads a maximum of **size - 1 characters**, leaving the final remaining slot open to append the null terminator \0 automatically. This makes it impossible to experience a buffer overflow exploit.

Comprehensive Output Handling: puts()

The puts() function is a specialized tool built specifically for displaying strings to the screen.

```
char name[30] = "Gaurav Kandpal";
puts(name); ----> Displays the string
```

Automatic Newline: Unlike printf("%s", name);, the puts() function automatically appends a newline character (\n) to the end of its output stream, saving you from writing extra positioning code.

Input/Output Method Matrix

Function	Reads Spaces?	Enforces Size Limits?	Best Used For
scanf()	✗ No	⚠ Only if specified	Quick, single-word string data entries.
gets()	Yes	✗ No	Legacy obsolete ; avoid entirely due to memory safety flaws.
fgets()	Yes	Yes	Industry standard for secure, multi-word textual entries.
printf()	N/A	N/A	Flexible, structured, and formatted output pipelines.
puts()	N/A	N/A	Quick string outputs that require a trailing newline.